TaskFlow

High-Level Design Document

Task Management Platform

Version 1.0 · April 24, 2026 · Confidential

Stack	Laravel 13 · PHP 8.3+ · Inertia.js · Vue 3 · MySQL · Tailwind CSS
Document Type	High-Level Design (HLD)
Status	Draft — Internal
Author	Hlaing

0 TABLE OF CONTENTS

1 PROJECT OVERVIEW

TaskFlow is a private, single-tenant task management web application built for individuals who need a structured Kanban-style workflow. Each registered user owns their own isolated workspace containing projects, boards, and tasks. A dedicated Super Admin interface provides platform-level control over all user accounts.

Attribute	Detail
Product Name	TaskFlow
Type	Web Application (SPA via Inertia.js)
Target Users	Individual users + 1 Super Admin
Data Isolation	Each user's data is fully private
Auth Method	Email + Password (custom implementation)
Version	1.0 — MVP

2 GOALS & NON-GOALS

Goals (V1)

- Provide secure, per-user authentication (register / login / logout / password reset)
- Allow each user to create, update, and delete multiple projects
- Implement a fully dynamic Kanban board per project with customisable column headers
- Support drag-and-drop task reordering and cross-column movement
- Store tasks with title, description/notes, due date, and priority level
- Expose a Super Admin dashboard with user management and impersonation capability
- Maintain clean, responsive UI using Tailwind CSS across desktop and mobile

Non-Goals (V1)

- No real-time collaboration — projects are strictly single-owner
- No member invitation or shared project access in V1
- No file attachments on tasks in V1
- No third-party OAuth (Google, GitHub) in V1
- No native mobile application — responsive web only
- No email notifications or reminders in V1

3 TECH STACK

Layer	Technology	Notes
Backend Framework	Laravel 13	PHP 8.3+ required
Frontend Framework	Vue 3 (Composition API)	SFC with <script setup>
Bridge	Inertia.js	Server-driven SPA, no separate API
Database	MySQL 8.x	Primary data store
Styling	Tailwind CSS v3	Utility-first, JIT mode
Drag & Drop	vue-draggable-plus / SortableJS	Kanban column & task DnD
Build Tool	Vite	Asset bundling, HMR
Authentication	Custom (no Breeze/Jetstream)	Full control, no overhead
ORM	Eloquent	Laravel built-in ORM
Queue / Jobs	Laravel Queue	For future async tasks
Testing	PHPUnit + Pest	Feature & unit tests

4 SYSTEM ARCHITECTURE

Overview

TaskFlow uses the Inertia.js monolith pattern — Laravel handles routing, auth, and business logic; Vue 3 renders the UI. There is no separate REST API consumed by the frontend; Inertia passes props directly from controllers to page components. This keeps the codebase unified while delivering a smooth SPA experience.

Browser (Vue 3 + Inertia)
↕ *Inertia.js XHR / full-page requests*
Laravel 13 (Routes → Middleware → Controllers → Models)
↕ *Eloquent ORM*
MySQL 8 Database

Key Architectural Decisions

- **Inertia.js monolith** — eliminates the need for a separate API layer, reducing complexity and token mismatches during development
- **Custom authentication** — gives full control over session handling, middleware, and future extensibility (e.g. 2FA, OAuth)
- **Role-based access** — a simple *is_admin* flag on the users table gates the Super Admin area via Laravel middleware
- **Optimistic UI** — drag-and-drop updates fire immediately on the frontend, with a backend sync call to persist order
- **Column ordering** — stored as an integer *position* field, updated in bulk on reorder events

5 DATABASE SCHEMA

Table: users

Column	Type / Notes
id	BIGINT PK AUTO_INCREMENT
name	VARCHAR(255)
email	VARCHAR(255) UNIQUE
password	VARCHAR(255) hashed
is_admin	BOOLEAN DEFAULT false
email_verified_at	TIMESTAMP NULL
remember_token	VARCHAR(100) NULL
created_at / updated_at	TIMESTAMPS

Table: projects

Column	Type / Notes
id	BIGINT PK AUTO_INCREMENT
user_id	BIGINT FK → users.id
name	VARCHAR(255)
description	TEXT NULL
created_at / updated_at	TIMESTAMPS

Table: columns (task_columns)

Column	Type / Notes
id	BIGINT PK AUTO_INCREMENT
project_id	BIGINT FK → projects.id
title	VARCHAR(100)
position	INT — ordering
created_at / updated_at	TIMESTAMPS

Table: tasks

Column	Type / Notes

id	BIGINT PK AUTO_INCREMENT
column_id	BIGINT FK → task_columns.id
project_id	BIGINT FK → projects.id
title	VARCHAR(255)
description	TEXT NULL
priority	ENUM(low, medium, high) DEFAULT medium
due_date	DATE NULL
position	INT — ordering within column
created_at / updated_at	TIMESTAMPS

Table: password_reset_tokens

Column	Type / Notes
email	VARCHAR(255) PK
token	VARCHAR(255)
created_at	TIMESTAMP NULL

6 FEATURE BREAKDOWN

Authentication

- Register with name, email, password (bcrypt hashed)
- Login with email + password, session-based auth
- Logout (invalidate session + regenerate token)
- Password reset via signed email link
- Remember me (extend session lifetime)
- Auth middleware guards all non-public routes

User Dashboard

- Overview of all projects (name, task count, last updated)
- Quick-create project button
- Per-user namespace — users never see each other's data

Project Management

- Create project (name + optional description)
- Edit project name / description
- Delete project (cascades to columns + tasks)
- List all projects with search/filter

Kanban Board

- Dynamic column headers — create, rename, delete, reorder columns
- Default columns on new project: To Do · Doing · Done
- Add tasks to any column
- Edit task (title, description, priority, due date)
- Delete task (with confirmation)
- Drag task within column (reorder) or to another column (status change)
- Column deletion moves tasks to first available column or prompts
- Visual priority indicators (colour-coded low / medium / high)
- Due date display with overdue highlighting

7 API ROUTE MAP

All routes are Inertia routes (returning Inertia responses, not JSON) unless noted. Prefix **/admin** routes are protected by the *IsAdmin* middleware.

Auth Routes

Method	Path	Description
GET	/login	Show login page
POST	/login	Authenticate user
POST	/logout	Destroy session
GET	/register	Show register page
POST	/register	Create user account
GET	/forgot-password	Show forgot password form
POST	/forgot-password	Send reset link email
GET	/reset-password/{token}	Show reset form
POST	/reset-password	Reset password

Dashboard

Method	Path	Description
GET	/dashboard	User dashboard (projects overview)

Projects

Method	Path	Description
GET	/projects	List all user projects
POST	/projects	Create project
GET	/projects/{project}	Show project Kanban board
PATCH	/projects/{project}	Update project details
DELETE	/projects/{project}	Delete project

Columns

Method	Path	Description
POST	/projects/{project}/columns	Create column

PATCH	/projects/{project}/columns/{column}	Rename column
DELETE	/projects/{project}/columns/{column}	Delete column
POST	/projects/{project}/columns/reorder	Bulk reorder columns

Tasks

Method	Path	Description
POST	/projects/{project}/tasks	Create task
PATCH	/projects/{project}/tasks/{task}	Update task
DELETE	/projects/{project}/tasks/{task}	Delete task
POST	/projects/{project}/tasks/reorder	Reorder / move task (DnD)

Super Admin

Method	Path	Description
GET	/admin/dashboard	Platform stats overview
GET	/admin/users	List all users
PATCH	/admin/users/{user}	Update user (ban, unban)
DELETE	/admin/users/{user}	Delete user account
POST	/admin/users/{user}/impersonate	Impersonate user
POST	/admin/impersonate/leave	Leave impersonation

8 SUPER ADMIN DASHBOARD

The Super Admin area is accessible only to users where *is_admin* = *true*. It is protected by a dedicated **IsAdmin** middleware applied to all */admin/** routes. The admin account is seeded via a DatabaseSeeder and is never self-registerable.

Admin Capabilities

- **Platform Stats** — total users, total projects, total tasks, new sign-ups (last 30 days)
- **User Listing** — paginated table of all users with name, email, join date, project count, status
- **Ban / Unban** — sets a *banned_at* timestamp; banned users are rejected at login middleware
- **Delete User** — permanently removes user and all cascaded data (projects, columns, tasks)
- **Impersonate User** — stores original admin ID in session, switches auth context to target user
- **Leave Impersonation** — restores original admin session cleanly

Security Notes for Impersonation

Impersonation stores *impersonating_id* in the session. All admin routes check that the original session identity holds the is_admin flag, preventing privilege escalation. A persistent banner is shown in the UI while impersonating. Impersonated actions are logged with both the admin ID and the target user ID.

9 SECURITY CONSIDERATIONS

Concern	Mitigation
CSRF Protection	Laravel's built-in CSRF middleware on all POST/PATCH/DELETE routes. Inertia sends the XSRF-TOKEN cookie automatically.
Password Hashing	bcrypt via Laravel's Hash facade. Raw passwords are never stored or logged.
Policy-based Authorisation	Laravel Policies (ProjectPolicy, TaskPolicy, ColumnPolicy) ensure users can only access their own resources. Gate checks in every controller method.
Mass Assignment Protection	All models define explicit $fillable arrays. $guarded = [] is never used.
SQL Injection	Eloquent parameterised queries throughout. No raw DB::statement with user input.
XSS	Inertia + Vue's template binding auto-escapes output. v-html is never used with user content.
Rate Limiting	Login endpoint rate-limited to 5 attempts / minute per IP via Laravel's RateLimiter.
Session Security	Session regenerated on login. Secure, HttpOnly, SameSite=Lax cookie flags.
Admin Route Guard	IsAdmin middleware terminates the request with 403 if the authenticated user is not an admin.

10 OPTIONAL FEATURES (V2+)

Feature	Effort	Notes
Bug Report / Feedback Form	MEDIUM	In-app form submitting to a feedback table. Admin can view/triage reports in the admin dashboard.
Task Comments	MEDIUM	Threaded comments per task. New task_comments table (task_id, user_id, body, timestamps).
Member Invitation & Project Access	HIGH	Invite users by email to a project with role (viewer / editor). Requires project_members pivot table and role-based policy updates.
Task Completion Reporting	LOW	Aggregate stats per project: tasks completed vs. pending, overdue rate, velocity chart. Exportable as CSV.
Email Notifications	LOW	Notify user of approaching due dates, completed tasks. Laravel Notifications + Mailable.
Google OAuth	LOW	Socialite driver for Google login, linked to existing accounts by email.

11 PROJECT FOLDER STRUCTURE

Laravel (Backend)

```
app/
  Http/
    Controllers/
      Auth/          ← RegisterController, LoginController, PasswordResetController
      ProjectController.php
      ColumnController.php
      TaskController.php
      Admin/
        UserController.php
        ImpersonateController.php
    Middleware/
      IsAdmin.php
      EnsureNotBanned.php
  Models/
    User.php  Project.php  Column.php  Task.php
  Policies/
    ProjectPolicy.php  ColumnPolicy.php  TaskPolicy.php
database/
  migrations/
  seeders/
    DatabaseSeeder.php   ← seeds super admin account
routes/
  web.php              ← all Inertia routes
```

Vue / Inertia (Frontend)

```
resources/js/
  Pages/
    Auth/        ← Login.vue  Register.vue  ForgotPassword.vue
    Dashboard.vue
    Projects/
      Index.vue  Create.vue  Edit.vue
    Board/
      Show.vue         ← Kanban board
      components/
        KanbanColumn.vue
        TaskCard.vue
        TaskModal.vue
    Admin/
      Dashboard.vue
      Users/  Index.vue  Show.vue
  Layouts/
    AppLayout.vue
    AdminLayout.vue
  Components/
    Shared/    ← Button, Modal, Badge, Dropdown …
  composables/
    useDragDrop.js
    useBoard.js
```

12 MILESTONES & ROADMAP

Phase	Focus	Timeline	Key Deliverables
Phase 1	**Foundation**	Week 1–2	• Laravel 13 project setup, Vite + Vue 3 + Inertia.js configured • MySQL database + migrations for users, projects, columns, tasks • Custom auth (register, login, logout, password reset) • Basic routing & middleware (auth guard, IsAdmin)
Phase 2	**Core Features**	Week 3–4	• User dashboard with project listing • Full CRUD for projects • Kanban board UI with static columns • Task CRUD (create, edit, delete with modal)
Phase 3	**Kanban Polish**	Week 5–6	• Dynamic column management (create, rename, delete, reorder) • Drag & drop via vue-draggable-plus (within column + cross-column) • Task priority badges and due date display + overdue highlighting • Optimistic UI updates for smooth DnD experience
Phase 4	**Admin Dashboard**	Week 7	• Super Admin layout and stats overview • User management (list, ban/unban, delete) • Impersonation feature with session-based switching • Admin-only route protection and audit logging
Phase 5	**QA & Launch**	Week 8	• Feature tests (PHPUnit/Pest) for all controller actions • Responsive UI review across breakpoints • Security review (CSRF, policies, rate limiting) • Deployment preparation (env config, production optimisations)